

TUPLAS EM PYTHON (, ,)

Guia completo com exemplos, resultados e explicações

Prof. Gustavo Franz (Science/Biology)

Python Developer in Progress

Building Educational Solutions with Python

GitHub: github.com/GustaFranz

Regras principais

- ✓ Tuplas são **IMUTÁVEIS**: não dá para alterar, adicionar ou remover elementos depois de criadas.
- ✓ Ordenadas: os elementos possuem posição (índice).
- ✓ Permitem elementos duplicados.
- ✓ Podem armazenar diferentes tipos de dados.
- ✓ Usamos parênteses () para criar tuplas.

Criando tuplas

Formas de criar uma tupla

```
t1 = (10, 20, 30) # tupla com numeros
t2 = ('Python', 3.14, True) # tipos diferentes
t3 = (1,) # 1 elemento (vírgula obrigatória)
t4 = tuple([1, 2, 3, 4]) # convertendo lista em tupla
```

Observação importante

(1) NAO é uma tupla — e apenas um valor entre parênteses. A tupla de um elemento exige a vírgula: (1,).

Operações e análise de tuplas

Exemplos considerando **numeros = (10, 20, 30, 40, 50)**.

Comando	Resultado	Por que esse é o resultado?
<code>numeros[0]</code>	10	O índice 0 é o primeiro elemento da tupla.
<code>numeros[2]</code>	30	O índice 2 aponta para o terceiro elemento.
<code>numeros[-1]</code>	50	Índices negativos começam pelo final: -1 é o último.
<code>numeros[-2]</code>	40	O -2 é o penúltimo elemento.
<code>numeros[1:4]</code>	(20, 30, 40)	Do índice 1 (incluído) até antes do 4.
<code>numeros[0:3]</code>	(10, 20, 30)	Os índices 0, 1 e 2, parando antes do 3.
<code>numeros[2:]</code>	(30, 40, 50)	Do índice 2 até o último elemento.

Comando	Resultado	Por que esse e o resultado?
<code>numeros[:3]</code>	<code>(10, 20, 30)</code>	Do primeiro ate antes do indice 3.
<code>numeros[:]</code>	<code>(10, ..., 50)</code>	Todos os elementos (copia da tupla).
<code>numeros[-3:]</code>	<code>(30, 40, 50)</code>	O indice -3 e o valor 30; vai ate o final.
<code>len(numeros)</code>	<code>5</code>	<code>len()</code> conta quantos elementos existem na tupla.
<code>30 in numeros</code>	<code>True</code>	<code>in</code> verifica se o valor 30 esta na tupla.
<code>100 in numeros</code>	<code>False</code>	O valor 100 nao existe na tupla.
<code>numeros.count(20)</code>	<code>1</code>	<code>count()</code> informa quantas vezes o valor aparece.
<code>numeros.index(40)</code>	<code>3</code>	<code>index()</code> retorna o indice da primeira ocorrencia.
<code>numeros + (60, 70)</code>	<code>(..., 60, 70)</code>	Cria uma nova tupla unindo as duas; a original nao muda.
<code>numeros * 2</code>	<code>(...repetida)</code>	Repete a sequencia inteira da tupla duas vezes.

Fatiamento (slice) visual

Considere `numeros = (10, 20, 30, 40, 50)` e seus indices:

Indice	0	1	2	3	4
Valor	10	20	30	40	50

Expressao	Resultado	Leitura
<code>numeros[1:4]</code>	<code>(20, 30, 40)</code>	Do indice 1 (incluido) ate antes do 4.
<code>numeros[2:]</code>	<code>(30, 40, 50)</code>	Do indice 2 ate o final.
<code>numeros[:3]</code>	<code>(10, 20, 30)</code>	Do inicio ate antes do indice 3.
<code>numeros[-3:]</code>	<code>(30, 40, 50)</code>	Os tres ultimos elementos.

Regra de ouro do fatiamento

O indice inicial e INCLUIDO; o indice final NAO e incluido.

Exercicio: medias e situacao dos alunos

Voce recebeu os dados de alunos e suas notas. Calcule a media de cada um e exiba nome, notas, media e situacao (Aprovado se media $\geq$ 7; senao Reprovado).

medias.py

```
alunos = (  
    ('Ana', (8.5, 7.0, 9.0)),  
    ('Bruno', (6.0, 5.5, 7.0)),  
    ('Carla', (9.5, 9.0, 8.5)),  
    ('Daniel', (7.0, 7.5, 6.5)),  
)  
  
resultado = []  
for nome, notas in alunos:  
    media = sum(notas) / len(notas)  
    situacao = 'Aprovado' if media >= 7 else 'Reprovado'  
    resultado.append((nome, round(media, 2), situacao))  
  
for nome, media, situacao in resultado:  
    print(nome, media, situacao)
```

Aluno	Notas	Media	Situacao
Ana	(8.5, 7.0, 9.0)	8.17	Aprovado
Bruno	(6.0, 5.5, 7.0)	6.17	Reprovado
Carla	(9.5, 9.0, 8.5)	9.00	Aprovado
Daniel	(7.0, 7.5, 6.5)	7.00	Aprovado

Dica final

Tuplas são ideais para dados que NAO devem mudar — como coordenadas, configurações e registros fixos.

Prof. Gustavo Franz (Science/Biology)

Python Developer in Progress

Building Educational Solutions with Python

Professor de Ciências e Biologia desde 2013 — atualmente estudando Programação.

Eu crio estes resumos para organizar os assuntos e estudar de forma clara e objetiva. Estou compartilhando porque eles também podem ajudar outros desenvolvedores que estão começando. Bons estudos — vamos aprender juntos!

GitHub: github.com/GustaFranz

Exercícios Python: github.com/GustaFranz/exercicios_python

Para quem quiser ver a resolução dos meus exercícios em Python.

EasyAnsi: github.com/GustaFranz/easyansi

Para quem quiser codificar personalizando com cores e estilos de forma mais fácil, prática e intuitiva.